

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Duration: 60 Mins | On Class | Total Marks: 50

Annexure A

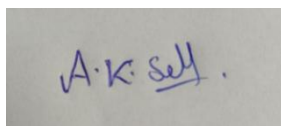
SIMULATION TASK

Code of Conduct:

I A K Selva Prabhu (192421224) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q. No	Conceptual Understanding (20)	Model Design (25)	Tool Usage (20)	Analysis of Results (20)	Documentation (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL	/ 4	/ 4	/ 4	/ 4	/ 4	
TOTAL	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100

Annexure A

SIMULATION TASK

Q1. A compiler receives an intermediate representation of an arithmetic expression containing multiple operators and temporary values.

Solution:

i. Simulate instruction selection for the arithmetic operations

The intermediate representation for the expression $w = (p + q) * r - (s - u)$ is:

```
t1 = p + q
t2 = t1 * r
t3 = s - u
t4 = t2 + t3
w = t4
```

Selecting appropriate target instructions for each IR statement:

IR Instruction	Target Instruction
t1 = p + q	ADD R1, p, q
t2 = t1 * r	MUL R2, R1, r
t3 = s - u	SUB R3, s, u
t4 = t2 + t3	ADD R2, R2, R3
w = t4	STORE w, R2

ii. Allocate registers for operands and temporary values

Assume $p = 6$, $q = 9$, $r = 4$, $s = 30$, $u = 8$.

Register	Holds
R1	t1 (p + q)
R2	t2 / t4 ((p+q) × r)
R3	t3 (s - u)

iii. Generate the corresponding target instruction sequence

```
LOAD R1, p
ADD R1, q
MOV R2, R1
MUL R2, r
LOAD R3, s
SUB R3, u
ADD R2, R3
STORE w, R2
```

iv. Show the register contents after each instruction

Step	Instruction	R1	R2	R3
1	LOAD R1, p	6	–	–
2	ADD R1, q	15	–	–
3	MOV R2, R1	15	15	–
4	MUL R2, r	15	60	–
5	LOAD R3, s	15	60	30
6	SUB R3, u	15	60	22
7	ADD R2, R3	15	82	22
8	STORE w, R2	15	82	22

Final result: w = 82

v. Evaluate the efficiency of the generated code

If every temporary value were written back to memory, each of t1–t4 would require a separate store and reload, adding four extra memory accesses. By keeping the temporaries in R1–R3:

- Only one final STORE is needed instead of four intermediate ones.
- Arithmetic operates directly on register operands, which is faster than memory-to-memory operations.
- Register R2 is reused for both t2 and t4, reducing the register count needed from four temporaries down to three registers.
- Overall instruction count and memory traffic are minimised, giving more efficient target code.

Python simulation

```
p, q, r, s, u = 6, 9, 4, 30, 8
registers = {"R1": None, "R2": None, "R3": None}

instructions = [
    "LOAD R1, p", "ADD R1, q", "MOV R2, R1", "MUL R2, r",
    "LOAD R3, s", "SUB R3, u", "ADD R2, R3", "STORE w, R2"
]

registers["R1"] = p; print(1, instructions[0], registers)
registers["R1"] += q; print(2, instructions[1], registers)
registers["R2"] = registers["R1"]; print(3, instructions[2], registers)
registers["R2"] *= r; print(4, instructions[3], registers)
registers["R3"] = s; print(5, instructions[4], registers)
registers["R3"] -= u; print(6, instructions[5], registers)
registers["R2"] += registers["R3"]; print(7, instructions[6], registers)
w = registers["R2"]; print(8, instructions[7], registers)
print("Final result: w =", w)
```

Output:

```
1 LOAD R1, p  {'R1': 6, 'R2': None, 'R3': None}
2 ADD R1, q   {'R1': 15, 'R2': None, 'R3': None}
3 MOV R2, R1  {'R1': 15, 'R2': 15, 'R3': None}
4 MUL R2, r   {'R1': 15, 'R2': 60, 'R3': None}
```

```
5 LOAD R3, s  {'R1': 15, 'R2': 60, 'R3': 30}
6 SUB R3, u   {'R1': 15, 'R2': 60, 'R3': 22}
7 ADD R2, R3  {'R1': 15, 'R2': 82, 'R3': 22}
8 STORE w, R2 {'R1': 15, 'R2': 82, 'R3': 22}
Final result: w = 82
```

Q2. A sequence of nested procedure invocations is represented in intermediate form.

Solution:

i. Simulate runtime stack creation

Consider the nested procedure invocation:

```
MAIN
  ↓
ALPHA(20)
  ↓
BETA(35)
```

Procedure ALPHA calls procedure BETA.

ii. Construct activation records

Each activation record stores the parameter, return address, control link and local variables.

MAIN AR	ALPHA AR	BETA AR
-----	-----	-----
Parameters : None	Parameter : x = 20	Parameter : y = 35
Return : OS	Return : MAIN	Return : ALPHA
Local : None	Control : MAIN AR	Control : ALPHA AR
	Local : k	Local : m

iii. Allocate storage for parameters and local variables

Assume each parameter and local variable occupies 4 bytes, and return address / control link each occupy 4 bytes.

Activation Record	Parameter	Return Address	Control Link	Local Variable	Total
ALPHA	4 bytes	4 bytes	4 bytes	4 bytes	16 bytes
BETA	4 bytes	4 bytes	4 bytes	4 bytes	16 bytes

iv. Show stack changes during procedure calls and returns

```
CALL MAIN
  Stack: [MAIN]

CALL ALPHA(20)
  Stack: [MAIN, ALPHA(x=20)]

CALL BETA(35)
  Stack: [MAIN, ALPHA(x=20), BETA(y=35)]

RETURN BETA
  Stack: [MAIN, ALPHA(x=20)]

RETURN ALPHA
  Stack: [MAIN]

RETURN MAIN
  Stack: [ ] (EMPTY)
```

v. Explain the role of runtime storage management during code generation

During code generation, the runtime storage manager:

- Allocates and deallocates activation records as procedures are called and return.
- Stores parameters, local variables and temporaries at the correct offsets in each frame.
- Maintains the return address so control can resume correctly in the calling procedure.
- Maintains the control (dynamic) link so nested and recursive calls can find the correct enclosing frame.
- Ensures each invocation gets its own storage, which is essential for correct recursion.

Python simulation

```
stack = []

def display_stack():
    print("Current Runtime Stack:", stack if stack else "EMPTY")

stack.append({"Procedure": "MAIN"}); print("CALL MAIN"); display_stack()
stack.append({"Procedure": "ALPHA", "Param": "x=20"}); print("CALL ALPHA(20)");
display_stack()
stack.append({"Procedure": "BETA", "Param": "y=35"}); print("CALL BETA(35)");
display_stack()
stack.pop(); print("RETURN BETA"); display_stack()
stack.pop(); print("RETURN ALPHA"); display_stack()
stack.pop(); print("RETURN MAIN"); display_stack()
```

Q3. Next-use information for variables is available before code generation begins.

Solution:

i. Construct the next-use table

Consider the intermediate instructions:

```
1. m = n + o
2. p = m + q
3. r = p * s
4. t = r + u
```

Statement	Variable	Next Use
1	m	2
1	n	None
1	o	None
2	p	3
2	q	None
3	r	4
3	s	None
4	t	None
4	u	None

ii. Simulate register allocation based on next-use information

Assume two available registers R1 and R2.

Statement	Operation	R1	R2
1	m = n + o	m	–
2	p = m + q	p	–
3	r = p * s	r	s
4	t = r + u	t	u

Register allocation chain: R1 → m → p → r → t | R2 → s / u

iii. Release registers whenever values become inactive

Variable	Last Use (Statement)	Action
n	1	Release
o	1	Release
m	2	Release
q	2	Release
p	3	Release
s	3	Release

Variable	Last Use (Statement)	Action
r	4	Release
u	4	Release

iv. Generate the final target instruction sequence

```
LOAD R1, n
ADD R1, o
ADD R1, q
LOAD R2, s
MUL R1, R2
LOAD R2, u
ADD R1, R2
STORE t, R1
```

v. Explain how next-use information improves register utilization

Next-use information tells the compiler exactly when a variable will be referenced again. If a variable has no further use, its register can be released and reused immediately rather than being kept occupied unnecessarily. This gives:

- Better utilisation of a limited number of registers.
- Fewer unnecessary memory loads and stores.
- Reduced register spilling to memory.
- A shorter, more efficient target instruction sequence.

Python simulation

```
next_use = {
    "m": 2, "p": 3, "r": 4,
    "n": None, "o": None, "q": None, "s": None, "u": None
}

for var, use in next_use.items():
    print(var, "->", use)

registers = {"R1": None, "R2": None}
registers["R1"] = "m"; print("Stmt 1:", registers)
registers["R1"] = "p"; print("Stmt 2:", registers)
registers["R1"], registers["R2"] = "r", "s"; print("Stmt 3:", registers)
registers["R2"] = "u"; print("Stmt 4 (before store):", registers)
registers["R1"] = "t"; print("Final:", registers)
```


Q4. An intermediate instruction sequence contains repeated computations inside a loop.

Solution:

i. Identify computations repeated during loop execution

Consider the intermediate instruction sequence:

```
for (i = 0; i < n; i++)
{
    t1 = x * y;
    t2 = t1 + i;
    z[i] = t2;
}
```

The computation $t1 = x * y$ is repeated on every iteration. Since x and y do not change inside the loop, $x * y$ is loop-invariant. For $n = 500$, this multiplication would otherwise be performed 500 times unnecessarily.

ii. Move loop-invariant computations outside the loop

Before optimisation:

```
for (i = 0; i < n; i++)
{
    t1 = x * y;
    t2 = t1 + i;
    z[i] = t2;
}
```

After optimisation:

```
t1 = x * y;
for (i = 0; i < n; i++)
{
    t2 = t1 + i;
    z[i] = t2;
}
```

iii. Generate the optimized loop

Before:

```
L1: t1 = x * y
    t2 = t1 + i
    z[i] = t2
    i = i + 1
    if i < n goto L1
```

After:

```
    t1 = x * y
L1: t2 = t1 + i
    z[i] = t2
    i = i + 1
    if i < n goto L1
```

iv. Compare the execution cost before and after optimization

Assume n = 500.

Operation	Before Optimization	After Optimization
Multiplication	500	1
Addition	500	500
Total Arithmetic Operations	1000	501
Operations Saved	–	499

v. Explain why loop optimization improves performance

Loop optimisation improves performance by moving computations whose operands do not change during loop execution outside the loop body. Benefits include:

- Fewer repeated calculations per iteration.
- Reduced CPU workload and instruction count.
- Shorter overall execution time.
- More efficient target code and lower power consumption.

Python simulation

```
n = 500
x, y = 7, 3

mult_before = add_before = 0
for i in range(n):
    t1 = x * y; mult_before += 1
    t2 = t1 + i; add_before += 1

mult_after = add_after = 0
t1 = x * y; mult_after += 1
for i in range(n):
    t2 = t1 + i; add_after += 1

print("Before:", mult_before + add_before)
print("After :", mult_after + add_after)
print("Saved :", (mult_before+add_before)-(mult_after+add_after))
```

Output:

```
Before: 1000
After : 501
Saved : 499
```

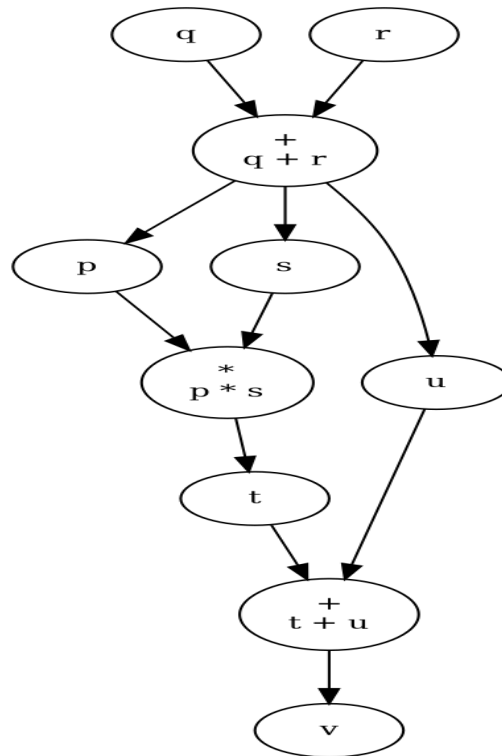
Q5. A basic block contains multiple repeated arithmetic expressions.

Solution:

i. Construct the Directed Acyclic Graph (DAG)

Consider the basic block:

```
p = q + r
s = q + r
t = p * s
u = q + r
v = t + u
```



ii. Identify common subexpressions

The repeated arithmetic expression is $q + r$, occurring three times:

- $p = q + r$
- $s = q + r$
- $u = q + r$

Therefore $q + r$ is identified as a common subexpression.

iii. Eliminate redundant computations

Original:

```
p = q + r
s = q + r
t = p * s
```

```
u = q + r
v = t + u
```

Evaluate $q + r$ only once:

```
t1 = q + r
p = t1
s = t1
t = p * s
u = t1
v = t + u
```

iv. Generate optimized intermediate instructions from the DAG

Since p , s and u all resolve to the same DAG node $t1$, the optimized intermediate code from the DAG is:

```
t1 = q + r
t = t1 * t1
v = t + t1
```

v. Compare the original and optimized instruction sequences

Parameter	Original	Optimized
Addition	4	2
Multiplication	1	1
Total Arithmetic Operations	5	3
Operations Saved	–	2
Reduction	–	40%

Python simulation

```
from graphviz import Digraph
dag = Digraph('DAG')

dag.node('q', 'q'); dag.node('r', 'r')
dag.node('plus1', '+\nq + r')
dag.edge('q', 'plus1'); dag.edge('r', 'plus1')

dag.node('p', 'p'); dag.node('s', 's'); dag.node('u', 'u')
dag.edge('plus1', 'p'); dag.edge('plus1', 's'); dag.edge('plus1', 'u')

dag.node('mul', '*\np * s')
dag.edge('p', 'mul'); dag.edge('s', 'mul')
dag.node('t', 't'); dag.edge('mul', 't')

dag.node('plus2', '+\nt + u')
dag.edge('t', 'plus2'); dag.edge('u', 'plus2')
dag.node('v', 'v'); dag.edge('plus2', 'v')
dag
```